SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
Student Name:	**Tharun**	**Reg. No.:**	**1911260024**
Date:	**26 October 2026**	**Duration:**	60 minutes

CO2 Statement: *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*

Code of Conduct

I **Tharun / 1911260024** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Tharun

1. SCENARIO STUDY AND ANALYSIS

Project Title: ProjectHub – Software Project Lifecycle Management Platform for Student Teams

1.1 Business Domain and Problem Context

Domain: Software engineering education and project management. Every semester, many teams of 3 to 5 students build a software project under a faculty mentor. Student software projects fail less from weak coding than from weak engineering process: requirements are scattered in chats, tasks are tracked in spreadsheets (or not at all), documents are overwritten, and mentors see real progress only at the final demo. ProjectHub gives each team one shared workspace that makes the software development life cycle (SDLC) visible, traceable and auditable.

1.2 Existing Challenges

- **Scattered requirements:** they live in chats and documents, and changes are never recorded.
- **No task tracking:** delays are noticed only at the deadline.
- **No visibility for mentors:** progress is judged from the final demo, not from continuous evidence.
- **Unfair grading:** individual contribution is hard to prove, which hides free-riders and penalises hard workers.
- **Lost versions:** documents are overwritten, and there is no link between a requirement, its code and its test.
- **Forgotten feedback:** review comments are given verbally and not followed up.

1.3 System Objectives

1. Give every team one shared workspace for requirements, tasks, documents and reviews.
2. Support iterative (Agile/Scrum) planning with backlog, sprints and a Kanban task board.
3. Provide requirement-to-task-to-test traceability and full version history.
4. Show real-time progress and individual contribution to mentors and the coordinator.
5. Automate reminders, reports and grading support using a rubric.
6. Keep all data secure, available and auditable.

1.4 Stakeholders

Stakeholder	Type	Needs from the system
Student / Team member	Primary user	Record requirements, work on tasks, share documents, show individual contribution.
Team Leader	Primary user	Create the team and project, plan sprints, assign work, submit deliverables.
Mentor (faculty)	Primary user	Review artefacts, approve requirements, monitor progress, grade fairly.
Course Coordinator / HoD	Primary user	Oversee all teams, define the grading rubric, see contribution reports.
System Administrator	Primary user	Manage accounts, security, backups and the audit log.
Institution / IT department	Sponsor	Low hosting cost, data privacy, policy compliance.
External evaluators	Secondary user	Read-only access to final deliverables.
Git hosting and email providers	External systems	Supply commit data and deliver notification emails.

2. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

2.1 Functional Requirements

36 functional requirements are grouped by module and prioritised with MoSCoW (M = Must, S = Should, C = Could).

ID	Requirement	Pri.
User and access management		
FR-01	The system shall let a new user register with name, institutional email, register number and role request.	M
FR-02	The system shall verify the email address before activating an account.	M
FR-03	The system shall authenticate users by email and password, with optional two-factor authentication.	M
FR-04	The system shall enforce role-based access so each role sees only its permitted functions and data.	M
FR-05	The administrator shall be able to create, edit, suspend and deactivate user accounts.	M
FR-06	The system shall let users reset a forgotten password through a time-limited email link.	M
Team and project management		
FR-07	A team leader shall be able to create a project with title, description, domain and dates.	M
FR-08	A leader shall be able to invite and remove team members (maximum 6 per team).	M
FR-09	The coordinator shall be able to assign a mentor to each team.	M
FR-10	The system shall support project phases: requirements, design, implementation, testing, deployment.	S
Requirements management		
FR-11	A member shall be able to add a requirement (functional or non-functional) with description, source, priority and acceptance criteria.	M
FR-12	The system shall keep a version history of every change to a requirement, with author and time.	M
FR-13	The mentor shall be able to approve or reject a requirement with a comment.	S
FR-14	The system shall flag requirements with empty acceptance criteria or duplicate descriptions.	C
FR-15	The system shall link each requirement to tasks, design documents and test cases (traceability).	M
Backlog, sprint and task management		
FR-16	The team shall be able to create user stories and tasks in a product backlog.	M
FR-17	The leader shall be able to create a sprint with start date, end date and goal, and move backlog items into it.	M
FR-18	The system shall provide a Kanban board with To Do, In Progress, In Review and Done columns.	M
FR-19	A task shall have an assignee, estimate, due date, priority and status.	M
FR-20	The system shall show a burndown chart for each sprint.	S
Documents and artefacts		
FR-21	A member shall be able to upload design documents, diagrams and reports (PDF, DOCX, PNG, MD) up to 25 MB.	M
FR-22	The system shall keep every file version and let users compare or restore versions.	M
FR-23	The system shall let users link a Git repository and import commits and pull requests.	S
Review, testing and quality		

ID	Requirement	Pri.
FR-24	A mentor shall be able to comment on any requirement, task or document.	M
FR-25	A mentor shall be able to open a review request and record the outcome (approved / changes needed).	M
FR-26	Members shall be able to reply to and resolve review comments.	S
FR-27	Members shall be able to record test cases against requirements and their pass/fail result.	M
FR-28	Members shall be able to log defects with severity, status and assignee.	S
Evaluation and reporting		
FR-29	The coordinator shall be able to define a grading rubric with criteria and weights.	M
FR-30	The mentor shall be able to score each team and each member against the rubric.	M
FR-31	The system shall report each member's contribution (tasks done, commits, reviews).	M
FR-32	The system shall generate a project status report as PDF.	S
Notifications and administration		
FR-33	The system shall send email and in-app alerts for assignments, due dates (24 h before) and review requests.	M
FR-34	The system shall show a role-based dashboard with progress, overdue tasks and upcoming deadlines.	M
FR-35	The administrator shall be able to back up and restore data and view the audit log.	M
FR-36	The system shall write an audit log entry for each create, update, delete and login event.	M

Summary: 28 Must, 7 Should, 1 Could.

2.2 Non-Functional Requirements

Each non-functional requirement carries a measurable target so that it can be tested.

ID	Category	Requirement	Measure / Target
NFR-01	Performance	Pages shall load quickly under normal load.	95% of pages < 3 s with 500 concurrent users
NFR-02	Performance	Search and dashboard queries shall respond fast.	< 2 s for 10,000 records
NFR-03	Security	Passwords stored hashed and salted; all traffic encrypted.	bcrypt or stronger; HTTPS (TLS 1.2+)
NFR-04	Security	Access follows RBAC; sessions expire; common web attacks resisted.	30 min idle logout; no OWASP Top-10 findings
NFR-05	Reliability	The system shall run without failure in normal use.	MTBF ≥ 500 h
NFR-06	Reliability	Data shall not be lost on failure.	Daily backup; recovery point 24 h, recovery time 4 h
NFR-07	Availability	Available during the academic term.	99.5% uptime excluding planned maintenance
NFR-08	Usability	A new student shall complete core tasks without training; accessible and responsive.	Create a task in < 2 min; WCAG 2.1 AA; 360–1920 px
NFR-09	Scalability	The system shall grow with more teams without redesign.	200 teams (1,000 users) now; 1,000 teams by adding servers
NFR-10	Maintainability	Code shall be modular, tested and documented.	≥ 70% unit-test coverage; REST API documented
NFR-11	Portability	The web client shall work on current browsers.	Latest 2 versions of Chrome, Edge, Firefox, Safari
NFR-12	Auditability	Changes to requirements and grades shall be traceable.	100% of changes logged with user and time
NFR-13	Privacy	Personal data used only for academic purposes.	Compliant with DPDP Act, 2023

3. ACTORS AND USE CASES

3.1 Actors

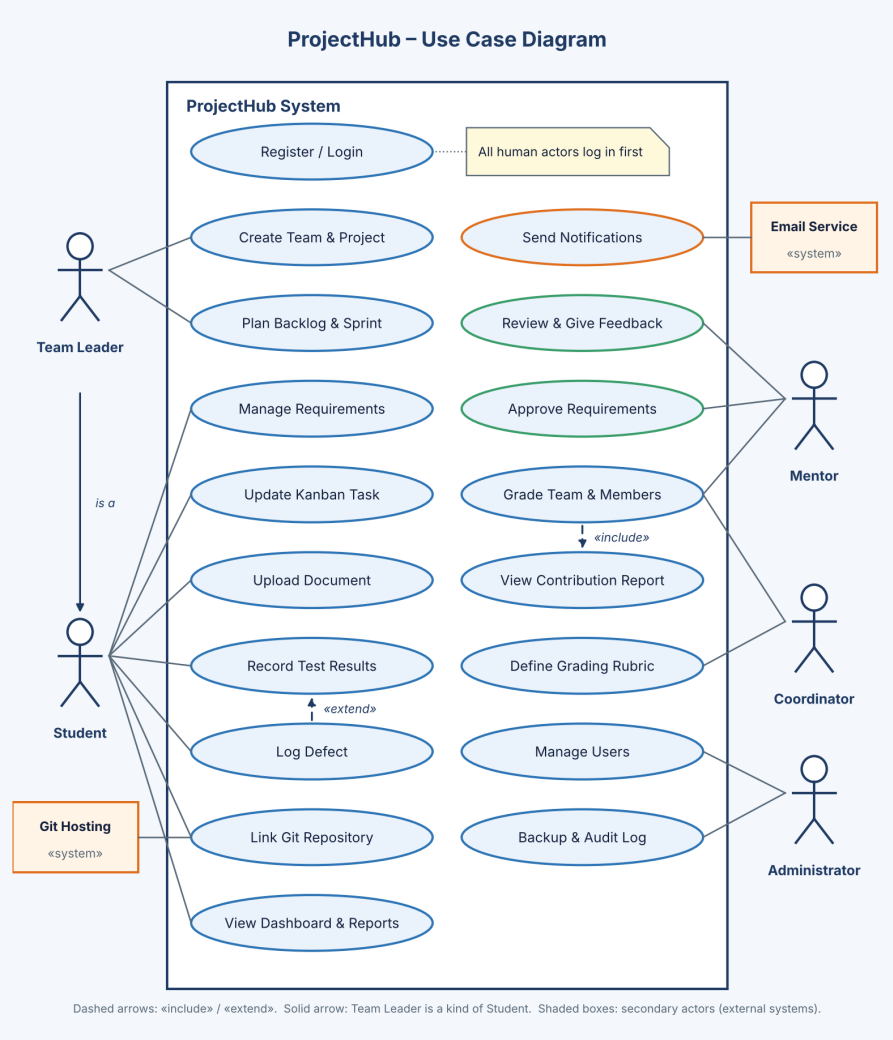
Actor	Type	Description
Student	Primary	Team member who records requirements, works on tasks, uploads documents and tests.
Team Leader	Primary	A Student who also creates the team and project and plans sprints (specialises Student).
Mentor	Primary	Faculty guide who reviews artefacts, approves requirements and grades.
Coordinator	Primary	Oversees all teams, sets the grading rubric, reviews contribution reports.
Administrator	Primary	Manages accounts, backups and the audit log.
Git Hosting Service	Secondary	External service (GitHub / GitLab) that supplies commit and pull-request data.
Email Service	Secondary	External SMTP provider that delivers notification emails.

3.2 Major Use Cases by Actor

ID	Use Case	Actor(s)	Related FR
UC-01	Register / Login	All human actors	FR-01 – FR-04, FR-06
UC-02	Manage Users	Administrator	FR-05
UC-03	Create Team & Project	Team Leader	FR-07 – FR-09
UC-04	Plan Backlog & Sprint	Team Leader	FR-16, FR-17, FR-20
UC-05	Manage Requirements	Student, Team Leader	FR-11, FR-12, FR-15
UC-06	Update Kanban Task	Student, Team Leader	FR-18, FR-19
UC-07	Upload Document	Student, Team Leader	FR-21, FR-22
UC-08	Record Test Results	Student, Team Leader	FR-27
UC-09	Log Defect (extends UC-08)	Student, Team Leader	FR-28
UC-10	Link Git Repository	Student, Team Leader, Git Hosting Service	FR-23
UC-11	View Dashboard & Reports	Student, Team Leader	FR-32, FR-34
UC-12	Send Notifications	Email Service (system-triggered)	FR-33
UC-13	Review & Give Feedback	Mentor	FR-24 – FR-26
UC-14	Approve Requirements	Mentor	FR-13
UC-15	Define Grading Rubric	Coordinator	FR-29
UC-16	Grade Team & Members (includes UC-17)	Mentor, Coordinator	FR-30
UC-17	View Contribution Report	Mentor, Coordinator	FR-31
UC-18	Backup & Audit Log	Administrator	FR-35, FR-36

3.3 Use Case Diagram

The diagram follows UML notation: actors outside the system boundary, use cases as ovals, solid lines for associations, a solid arrow for generalisation and dashed arrows for «include» and «extend» relationships.



4. SOFTWARE REQUIREMENT SPECIFICATION (SRS)

4.1 Introduction

Purpose: This SRS defines what ProjectHub must do and how well it must do it, giving students, mentors, developers and testers one agreed baseline. **Audience:** project teams, mentors and coordinators, administrators, developers and testers. **Conventions:** requirements are numbered FR-xx and NFR-xx, use cases UC-xx; priorities follow MoSCoW. **References:** IEEE 830 / ISO/IEC/IEEE 29148; Sommerville, *Software Engineering*; Pressman, *Software Engineering: A Practitioner's Approach*.

4.2 Problem Description

Student teams manage requirements, tasks, documents and reviews in separate tools, so progress is invisible, versions are lost and individual contribution cannot be proved. ProjectHub is a single web platform where a team plans sprints, records versioned requirements linked to tasks and tests, shares documents, receives mentor reviews, and where mentors and the coordinator see live progress and grade with a rubric.

4.3 Scope of the System

In scope	Out of scope (this release)
Registration, login and role-based access Team and project creation Requirements with versions and traceability Backlog, sprints and Kanban board Document upload with version history Review and feedback workflow Test cases and defect logging Git commit/PR import, notifications, dashboards, reports, rubric grading	Hosting or building student code (no built-in CI/CD) Video conferencing and live chat Native mobile apps (responsive web only) Automated plagiarism detection Payment processing

4.4 Use Case Descriptions

UC-01 Register / Login

Field	Description
Actor	Student, Team Leader, Mentor, Coordinator, Administrator
Precondition	User has an institutional email address.
Main flow	1. User chooses Register or Login. 2. For registration, user enters name, email, register number and role; system sends a verification link. 3. User verifies the email; account becomes active. 4. User enters email and password (and 2FA code if enabled). 5. System validates credentials and opens the role's dashboard.
Alternate flows	A1: Wrong password – error shown; after 5 failures the account locks for 15 min. A2: Forgotten password – user requests a reset link by email.
Postcondition	User has an authenticated session with role-based permissions.

UC-03 Create Team & Project

Field	Description
Actor	Team Leader
Precondition	Leader is logged in and is not already leading a project this term.
Main flow	1. Leader selects New Project. 2. Enters title, description, domain, start and end dates. 3. Invites members by email (up to 6). 4. System creates the workspace and notifies members. 5. Coordinator assigns a mentor.
Alternate flows	A1: Invited email has no account – system sends a registration invitation. A2: Team size exceeds 6 – invitation rejected.
Postcondition	A project workspace exists with its team and mentor.

UC-05 Manage Requirements

Field	Description
Actor	Student, Team Leader (Mentor approves via UC-14)
Precondition	Project exists; user is a member.
Main flow	1. Member chooses Add Requirement. 2. Enters description, type, source, priority and acceptance criteria. 3. System saves it as version 1 with status Draft. 4. Member links it to tasks, documents and test cases. 5. Member submits it for mentor approval.
Alternate flows	A1: Acceptance criteria empty – system warns the requirement is ambiguous. A2: Editing an approved requirement – system creates a new version and resets status to Draft.
Postcondition	Requirement is stored with full version history and traceability links.

UC-04 Plan Backlog & Sprint

Field	Description
Actor	Team Leader
Precondition	At least one approved requirement exists.
Main flow	1. Leader creates user stories from requirements. 2. Orders the backlog by priority. 3. Creates a sprint with goal and dates. 4. Moves stories into the sprint and assigns tasks with estimates. 5. System opens the Kanban board and burndown chart.
Alternate flows	A1: Sprint load exceeds team capacity – system warns the leader.
Postcondition	An active sprint with assigned, estimated tasks exists.

UC-13 Review & Give Feedback

Field	Description
Actor	Mentor
Precondition	Team has submitted a requirement, task or document for review.
Main flow	1. Mentor opens the review request. 2. Reads the artefact and adds comments. 3. Marks the outcome Approved or Changes Needed. 4. System notifies the team (UC-12). 5. Team replies and resolves comments.
Alternate flows	A1: Mentor does not respond within 3 days – system reminds the mentor and informs the coordinator.
Postcondition	Review outcome and comments are stored against the artefact.

UC-16 Grade Team & Members

Field	Description
Actor	Mentor, Coordinator
Precondition	Rubric defined (UC-15); project phase closed.
Main flow	1. Mentor opens the grading page. 2. System shows the contribution report (UC-17). 3. Mentor scores the team and each member on every rubric criterion. 4. System calculates weighted totals. 5. Mentor submits; coordinator reviews and publishes.
Alternate flows	A1: A criterion is left blank – system blocks submission. A2: Coordinator disagrees – grade returns to the mentor with a comment.
Postcondition	Final grades are locked and logged in the audit trail.

4.5 Data Requirements

Entity	Key attributes	Relationships
User	user_id (PK), name, email (unique), password_hash, register_no, role, status	A user belongs to many teams
Team	team_id (PK), name, leader_id (FK), mentor_id (FK)	Many members; one project
Project	project_id (PK), team_id (FK), title, description, domain, start_date, end_date, phase	Many requirements, sprints, documents
Requirement	req_id (PK), project_id (FK), type, description, priority, acceptance_criteria, status, version	Many versions, tasks, test cases
Sprint	sprint_id (PK), project_id (FK), goal, start_date, end_date	Many tasks
Task	task_id (PK), sprint_id (FK), req_id (FK), assignee_id (FK), title, estimate_h, due_date, status	One sprint, one assignee
Document / DocumentVersion	doc_id (PK), project_id (FK), name; version_id (PK), version_no, file_path, size_kb	A document has many versions
Review	review_id (PK), target_type, target_id, mentor_id (FK), outcome, created_at	Many comments
TestCase / Defect	test_id (PK), req_id (FK), result; defect_id (PK), test_id (FK), severity, status	A test case may have many defects
Rubric / Grade	rubric_id (PK), criterion, weight; grade_id (PK), user_id (FK), rubric_id (FK), score	One grade per member per criterion
AuditLog	log_id (PK), user_id (FK), action, entity, timestamp, ip_address	Written on every create, update, delete, login

- Email and register number are unique per user. Requirement, document and grade changes keep earlier versions; nothing is overwritten.
- Project data is kept for 5 years after the term; audit logs for 2 years. User accounts are deactivated, not deleted, so academic records stay intact.
- Uploaded files are limited to 25 MB and to PDF, DOCX, PNG, JPG and MD types. Estimated storage is about 4 GB per term for 200 teams.

4.6 External Interface Requirements

Interface	Requirement
User interface	Responsive web interface with a role-based dashboard, left navigation menu and top bar with notifications. Main screens: login, dashboard, requirements, backlog, sprint board (drag and drop), documents, reviews, tests and defects, grading, reports, admin console. Keyboard navigation and screen-reader labels (WCAG 2.1 AA).
Hardware	Any device with a modern browser. Server: Linux VM with at least 4 vCPU, 8 GB RAM and 200 GB storage.
Software	PostgreSQL database; object storage or file system for documents; Git hosting API (GitHub / GitLab, OAuth, REST/JSON); SMTP for email; PDF generator for reports; optional LDAP/OAuth directory to verify student emails.
Communication	All client-server traffic over HTTPS (TLS 1.2+). Internal REST API with JSON under /api/v1 protected by session tokens. Emails sent asynchronously through a queue.

4.7 System Features

#	Feature	Description	Priority
1	Secure access	Registration, login, 2FA, role-based permissions	High
2	Team and project workspace	Shared space with members, mentor and phases	High
3	Requirements and traceability	Versioned requirements linked to tasks, documents and tests	High
4	Agile planning board	Backlog, sprints, Kanban board, burndown chart	High
5	Document and Git integration	Versioned uploads and imported commit activity	Medium
6	Review and feedback	Threaded comments, approvals, review requests	High
7	Evaluation and contribution tracking	Rubric grading and per-member contribution report	High
8	Notifications, dashboards, administration	Alerts, role dashboards, reports, backup, audit log	Medium

5. ASSUMPTIONS AND CONSTRAINTS

5.1 Assumptions

- Every student has an institutional email address and a device with a modern browser and internet access.
- Each team has 3 to 6 members and exactly one mentor.
- Teams keep their code on a Git hosting service; ProjectHub only reads activity from it.
- The institution provides a server or cloud account and an SMTP service.
- The coordinator supplies the grading rubric before the project starts.
- Teams follow an iterative (Scrum-style) process with sprints of 1 to 4 weeks.
- The interface is English only in this release.

5.2 Constraints

Type	Constraint
Technical	Web-based only; open-source stack (PostgreSQL, Linux); must work on current versions of four major browsers; external integrations limited to Git hosting APIs and SMTP.
Operational	Used mainly during the academic term with peaks near review and submission dates; needs a trained administrator; maintenance outside exam periods.
Economic	Budget limited to institutional resources; no paid licences; a single small cloud instance in year one.
Legal	Digital Personal Data Protection Act, 2023; student data used only for academic purposes; open-source licences respected; WCAG 2.1 AA accessibility.
Time	Prototype in 8 weeks, full release in one semester (about 16 weeks), ready before the next term.
Organisational	Mentor and coordinator workflow must match the institution’s evaluation policy.

6. REQUIREMENT VALIDATION

6.1 Coverage of Stakeholder Requirements

Stakeholder need	Objective	Covered by
Requirements recorded and changes tracked	1, 3	FR-11 – FR-15; UC-05
Task tracking and early warning of delays	2, 5	FR-16 – FR-20, FR-33, FR-34; UC-04, UC-06, UC-12
Mentor can see real progress	4	FR-23, FR-31, FR-34; UC-10, UC-11, UC-17
Individual contribution can be proved	4	FR-23, FR-30, FR-31; UC-16, UC-17
No lost document versions	3	FR-21, FR-22; UC-07
Review feedback is recorded and followed up	1, 4	FR-24 – FR-26; UC-13
Security, availability, auditability	6	FR-35, FR-36; NFR-03 – NFR-07, NFR-12; UC-18

6.2 Quality Checks and Findings

Check	Finding	Improvement made / suggested
Ambiguity	NFR-01 and NFR-02 describe performance with “quickly” and “fast”; FR-14 does not define “ambiguous”.	Each has a measure column as the only test; ambiguous = empty acceptance criteria or vague words such as *some*, *fast*, *etc.*
Inconsistency	Deleting users (FR-05) conflicts with keeping records for 5 years.	FR-05 changed to deactivate; personal data anonymised on request, academic records kept.
Inconsistency	NFR-09 sizes the system for 200 teams but storage was first estimated for 100.	200 teams used everywhere; storage about 4 GB per term.
Redundancy	FR-12, FR-22 and FR-36 all record change history.	Kept: FR-36 is the security trail; FR-12 and FR-22 are user-visible history, sharing one versioning component.
Missing	No export of a project at term end.	Suggested FR-37: coordinator exports requirements, tasks and documents as ZIP and CSV.
Missing	No profile management or notification preferences.	Suggested FR-38: user edits profile and chooses which alerts arrive by email.
Missing	FR-10 (project phases) has no use case.	Suggested UC-19 Change Project Phase for the Team Leader.

6.3 Conclusion

Every stakeholder need in Section 1 is traced to at least one functional requirement and one use case, except FR-10 (to be covered by UC-19). Every non-functional requirement has a measurable target, and assumptions and constraints are stated. The Git integration (third-party API limits) and the 99.5% availability target (institution’s hosting) should be confirmed with stakeholders before design begins.

End of Document — This submission addresses all six required aspects of the Scenario-Based Requirement Analysis: scenario study, functional and non-functional requirements, actors and use cases with diagram, the SRS, assumptions and constraints, and requirement validation.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	**/25**